NEW YORK INSTITUTE OF TECHNOLOGY

College of Engineering & Computing Sciences

COURSE NAME: COMPUTATIONAL ANALYSIS

COURSE CODE: MATH330-M01

SEMESTER: SPRING 2026

Project Three

Polynomial Interpolation in Mathematica

AUTHOR: JASMINE TUIACHIEVA

INSTRUCTOR: DR. ANDREW HOFSTRAND

SUBMISSION DATE: 02-24-2026 (MM-DD-YYYY)

TABLE OF CONTENTS

ABSTRACT

This project examines several numerical techniques for data **interpolation** using the Mathematica platform. We begin by evaluating global polynomial interpolation through **Vandermonde** and **Lagrange** formulations, specifically focusing on how different functions such as the non-differentiable or the oscillatory functions respond to increasing polynomial degrees. We then derive the **Newton form** of the interpolating polynomial to demonstrate its recursive efficiency in handling dynamic datasets. Finally, the project concludes with the implementation of **Natural Cubic Splines**. By constructing a **tri-diagonal** system to solve for **piecewise cubic segments**, we demonstrate that spline interpolation offers a far more stable and reliable fit for complex data than high-degree global polynomials.

PART I: INTERPOLATING FUNCTIONS AT EQUALLY SPACED POINTS

1.1 SETUP

To initiate the project, I configured Mathematica to handle the indeterminate form of 0^0 by unprotecting the **`Power[]`** function, ensuring our Vandermonde matrix calculations remained consistent. I tested three distinct functions:

- $f(x) = e^x, \quad x \in [-1, 4]$
- $f(x) = |x|, \quad x \in [-1, 1]$
- $f(x) = \frac{1}{1+x^2}, \quad x \in [-5, 5]$

```
Unprotect[Power];
Power[0 | 0., 0 | 0.] = 1;
Protect[Power];
```

1.2 VANDERMONDE AND LAGRANGE IMPLEMENTATION

I defined three functions on specified intervals. The code utilizes a **`While[]`** loop to process each function.

```
(*Functions and their intervals*)
func = {E^x, Abs[x], 1/(1 + x^2)};
interv = {{-1, 4}, {-1, 1}, {-5, 5}};
n = 10;
i = 1;

(*A loop to determine the Vandermonde and Lagrange Polynomials for a
list of functions defined above*)
While[i <= Length[func],
```

```
 f[x_] = func[[i]];
 {a, b} = interv[[i]];
```

I partitioned each interval into n+1 points (n = 10) using the **`Table[]`** function.

```
(*Step 1: Partitioning the interval to n+1 equally spaced points*)
pointsXval = Table[a + (b - a) j/n, {j, 0, n}];(* j iterator starts
from 0 since we have to include a(the left endpoint) *)


pointsYval = Table[f[pointsXval[[k]]], {k, 1, n + 1}]; (* k iterator
starts from 1 since indexing in Mathematica starts from 1*)
```

I built the Vandermonde matrix V where each entry is $(x_i)^{j-1}$. The coefficients were found by inverting the matrix: **`coeffs = Inverse[V] . pointsYval`**.

```
(*Step 2: Computing Vandermonde matrix*)
V = Table[pointsXval[[iR]]^(jC - 1), {iR, 1, n + 1}, {jC, 1, n +
1}];(* i is the outermost. The matrix entries are essentially
(x_i)^(j-1)*)


coeffs = Inverse[V] . pointsYval; (*Performing Dot Multiplication, not
* (element-by-element)*)
```

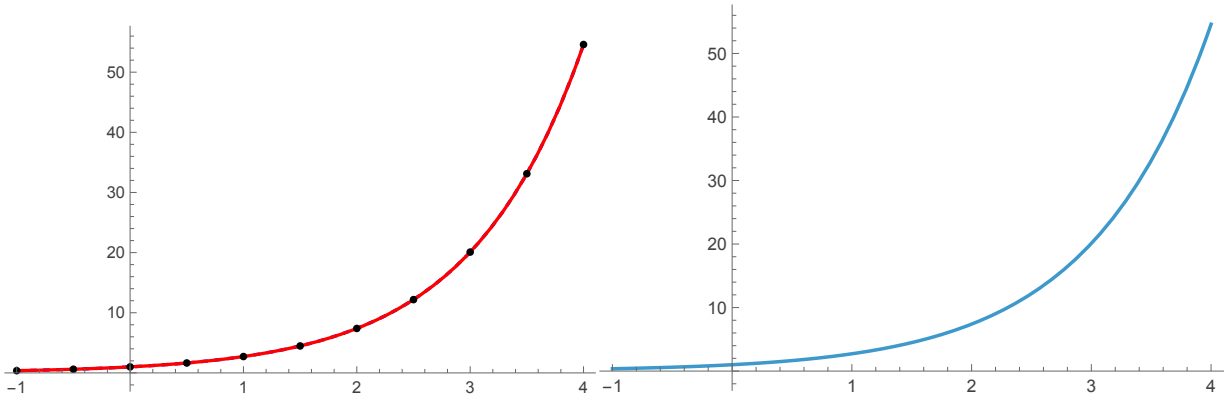
The polynomial $P_n(x)$ was constructed as a **power sum**, while was built using a **`Product[]`** loop to generate the Lagrange basis functions. This allowed to visually confirm that both methods result in the exact same interpolating curve.

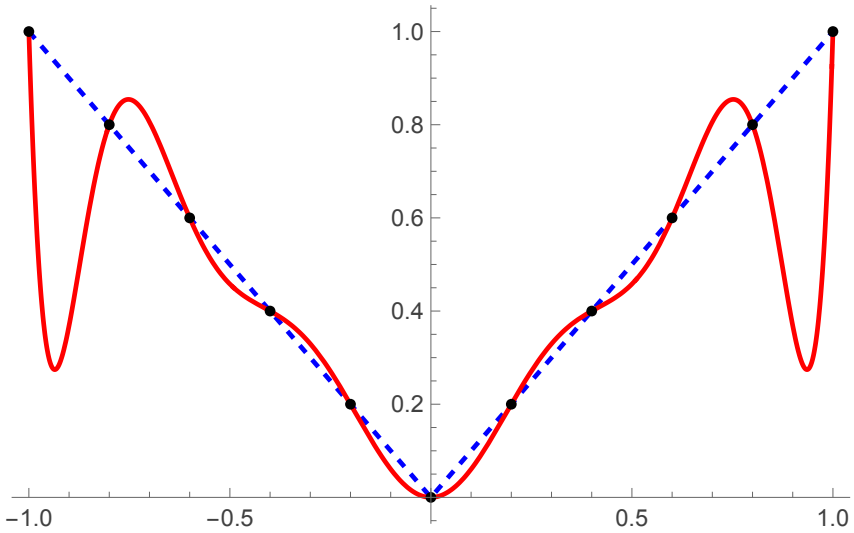
```
(*Step 3: Defining Pn (x) using the coefficients from Vandermode
matrix*)
Pn[x_] := Sum[coeffs[[j + 1]] x^j, {j, 0, n}];

(*Step 4: Defining Ln(x)*)
Li[k_, x_] = Product[
   If[m == k,
    1,
    (x - pointsXval[[m]])/(pointsXval[[k]] - pointsXval[[m]])
    ],
   {m, 1, n + 1}
   ];

Ln[x_] := Sum[pointsYval[[k]] Li[k, x], {k, 1, n + 1}];
```

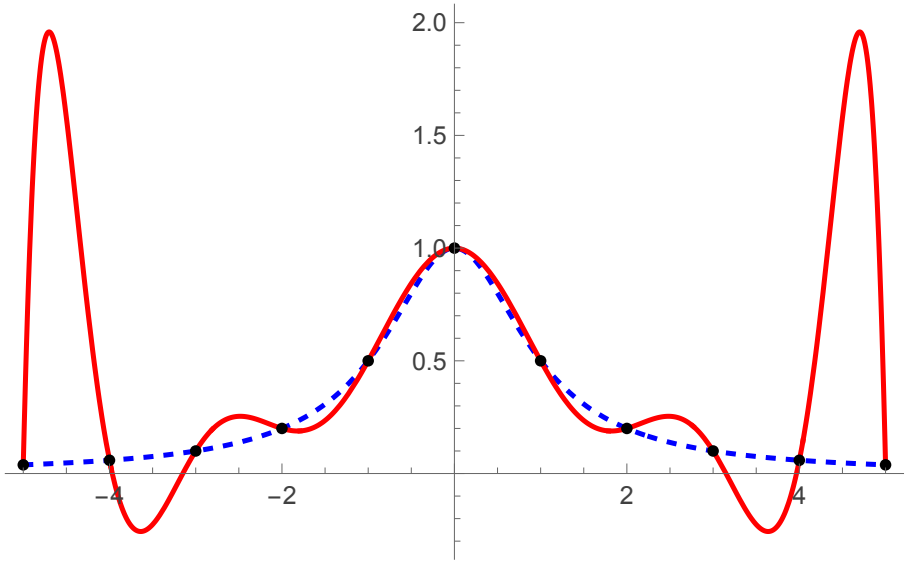
Let's **`Plot[]`** the results and close the **`While[]`** loop.

```
 (*Plotting results*)
 pointsPlot = ListPlot[Transpose[{pointsXval, pointsYval}], PlotStyle
```

```
-> Black];
 mainPlot = Plot[{f[x], Ln[x]}, {x, a, b}, PlotStyle -> {{Blue,
Dashed}, Red}];
 Print[Show[mainPlot, pointsPlot]]
 i++;
 ]
```







1.3 FINDINGS

- **Exponential (e^x)** : The fit was perfect. Since the function is smooth and its derivatives are bounded, the error decreases as n increases.
- **Absolute Value ($|x|$):** The interpolated function failed to match the original function at x = 0. Because the function is not differentiable at the origin, it violates the requirements for the error bounds in Equation 23.

$$E_n(t) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(t - x_0)(t - x_1)\cdots(t - x_n). \qquad (23)$$

- **Runge Function ($\frac{1}{1+x^2}$):** This function showcased Runge's Phenomenon. With equally spaced nodes, the polynomial developed extreme oscillations near the interval boundaries, demonstrating that higher-degree polynomials can decrease accuracy.

PART II: NEWTON FORM OF POLYNOMIAL INTERPOLATION

2.1 RECURSIVE CONSTRUCTION

The third form of interpolation, $W_n(x)$, is building the polynomial term-by-term. The first three terms are:

$$W_2(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1)$$

Each term k contains a product of all previous nodes $(x - x_0)$ … … … $(x - x_{k-1})$. This ensures that when we evaluate the polynomial at a previous node, the new term becomes zero, "protecting" the already established interpolation.

$$W(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1)$$

1st term: $W_0(x_0) = c_0 \Rightarrow y_0 = c_0$

2nd term: at $x = x_0$ $\quad W_1(x_0) = c_0 + c_1(x_0 - x_0)$ ($x_0 - x_0 = 0$)

$$W_1(x_0) = c_0$$

first term not affected

at $x = x_1$ $\quad W_1(x_1) = c_0 + c_1(x_1 - x_0)$

$$y_1 = c_0 + c_1(x_1 - x_0)$$

3rd term:

at $x = x_0$ $\quad W_2(x_0) = c_0 + c_1(x_0 - x_0) + c_2(x_0 - x_0)(x_0 - x_1)$

$$W_2(x_0) = c_0$$

$y_0 = c_0$ → first term not affected

at $x = x_1$

$$W_2(x_1) = c_0 + c_1(x_1 - x_0) + c_2(x_1 - x_0)(x_1 - x_1)$$

($x_1 - x_1 = 0$)

$y_1 = c_0 + c_1(x_1 - x_0)$

→ 2nd term not affected

at $x = x_2$ $\quad w_2(x_2) = c_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1)$

$y_2 = c_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1)$

4th term:

$$w_3(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + c_3(x - x_0)(x - x_1)(x - x_2)$$

5th term:

$$w_4(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + c_3(x - x_0)(x - x_1)(x - x_2) + c_4(x - x_0)(x - x_1)(x - x_2)(x - x_3)$$

Formula for general k^{th} term in $w(x)$:

$$c_k \prod_{j=0}^{k-1} (x - x_j)$$

2.2 COMPARISON WITH VANDERMONDE OR LAGRANGE INTERPOLATION

The practical value of the Newton form is its **flexibility**. If we receive a new data point, we do not need to recalculate the entire system as we would with Vandermonde or Lagrange method, we simply calculate one new coefficient c_{n+1} and add one term.

PART III: NATURAL CUBIC SPLINES

3.1 DATA PREPARATION AND NODE DISTANCES

The first step was to load the external dataset and visualize the raw information. I used **`ToExpression[]`** to ensure the data from the text file was treated as a mathematical list.

```
(*Step 1: Importing data and plotting bare points*)
data =
ToExpression[Import["/Users/jasminet/Downloads/data_splines.txt","List
"]];
pointsPlot = ListPlot[data, PlotStyle -> Black];

(*Step 2: Extracting x and y coordinates and computing widths h*)
n = Length[data];
x = data[[All, 1]];
y = data[[All, 2]];
h = Table[x[[j + 1]] - x[[j]], {j, 1, n - 1}];
```

I defined h_j as the horizontal distance between consecutive points.

3.2 CONSTRUCTING THE TRIDIAGONAL SYSTEM

Following Equations 30 and 31 from the project description, I calculated the components for the tridiagonal matrix. Note that to simplify the math in Mathematica, I multiplied the equations by their common denominators to clear the fractions.

$$U_j c_{j-1} + Q_j c_j + R_j c_{j+1} = V_j, \tag{30}$$

where

$$Q_j = \frac{2}{3}\left(h_j h_{j+1} + h_{j+1}^2\right), \qquad R_j = \frac{1}{3}h_{j+1}^2 \tag{31}$$

$$U_j = \frac{1}{3}h_{j+1}h_j, \qquad V_j = y_{j+1} - y_j + \frac{h_{j+1}(y_{j-1} - y_j)}{h_j}.$$

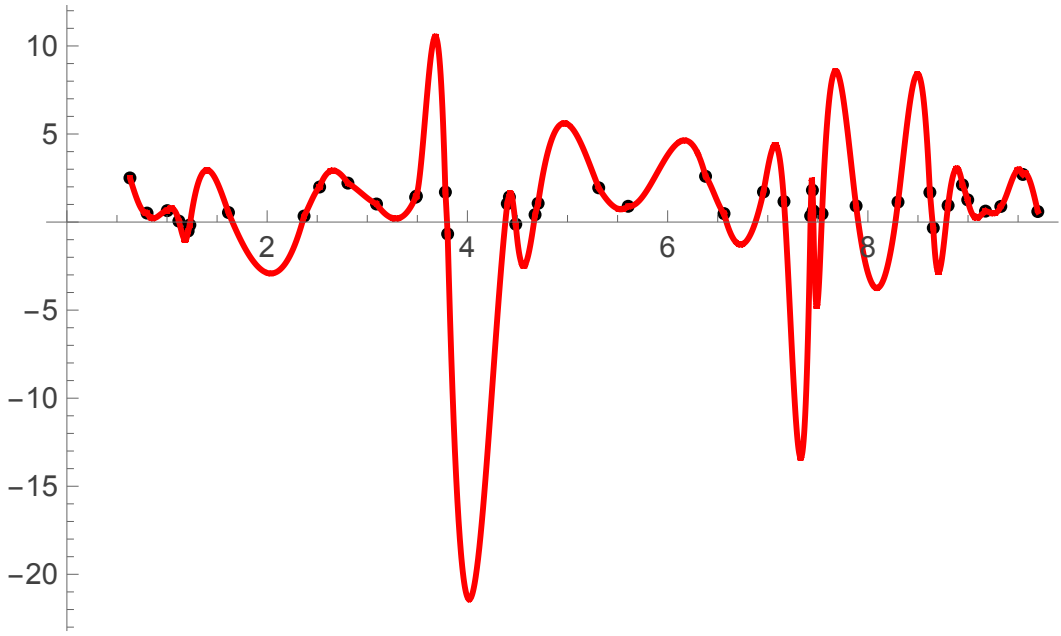
```
(*Step 3: Constructing matrix as per Equations 30 and 31*)
Vlist = Table[6*((y[[j + 2]] - y[[j + 1]])/h[[j + 1]] - (y[[j + 1]]-
y[[j]])/h[[j]]), {j, 1, n - 2}];
Qlist = Table[2*(h[[j]] + h[[j + 1]]), {j, 1, n - 2}];
Rlist = Table[h[[j + 1]], {j, 1, n - 3}];
Ulist = Table[h[[j + 1]], {j, 1, n - 3}];
```

To solve for the second derivative coefficients (c), I used a **SparseArray[]** with the **Band[ ]** command. I then applied the "Natural Spline" boundary condition by setting $\boldsymbol{c_0 = c_{n-1} = 0}$.

```
(*Step 4: Building the Tridiagonal matrix (Eq 32) and solving for c*)
matrix = SparseArray[{Band[{1, 1}] -> Qlist, Band[{1, 2}] ->
Rlist,Band[{2, 1}] -> Ulist}];


cInternal = Inverse[matrix] . Vlist;

(*Step 5: Natural spline boundary condition: c0=c_n-1=0*)
c = Join[{0}, cInternal, {0}];
```

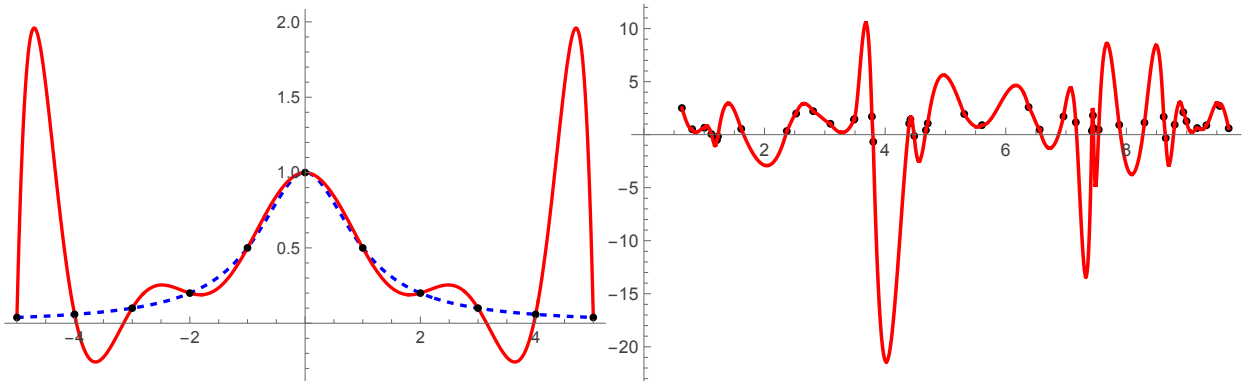
With the c coefficients found, I solved for b_j and d_j using Equations 28 and 29. Finally, I used a **While[]** loop to define a unique cubic polynomial $p_j(t)$ for every interval between the points.

```
(*Step 6: Computing coefficients b and d using Equations 28 and 29*)
b = Table[(y[[j + 1]] - y[[j]])/
     h[[j]] - (h[[j]]/3)*(c[[j + 1]] + 2*c[[j]]), {j, 1, n - 1}];
d = Table[(c[[j + 1]] - c[[j]])/(3*h[[j]]), {j, 1, n - 1}];

(*Step 7: Defining Piecewise polynomials and plotting in Red*)
plots = {pointsPlot};
j = 1;
While[j < n,
  pj[t_] :=
   y[[j]] + b[[j]]*(t - x[[j]]) + c[[j]]*(t - x[[j]])^2 +
    d[[j]]*(t - x[[j]])^3;
  AppendTo[plots,
   Plot[pj[t], {t, x[[j]], x[[j + 1]]}, PlotStyle -> Red]];
  j++;
  ];

Show[plots, PlotRange -> All]
```



3.3 FINDINGS

When I used a global polynomial P_{10} for the Runge function ($\frac{1}{1+x^2}$) in Part I, the curve oscillated wildly at the edges of the interval. However, the Cubic Spline method stays stable. Because the splines only use 3rd degree polynomials locally for each segment, they don't "overreact" to distant points.



ACKNOWLEDGEMENTS

I would like to express my gratitude to Dr. Andrew Hofstrand for his guidance and for providing the theoretical framework necessary to complete this project.

REFERENCES

PRIMARY COURSE DOCUMENTATION

Hofstrand, A. (2026). MATH330: Computational Analysis Course Notes on Numerical Interpolation and Spline Theory. New York Institute of Technology. [Referenced for Equation 23 (Error Bounds), Equations 28–29 (Coefficient Back solving), and Equations 30–32 (Tridiagonal Matrix Systems)].

DATA SOURCES

NYIT Department of Mathematics. (2026). data_splines.txt. [Dataset containing 41 coordinate pairs for piecewise cubic spline implementation].

MATHEMATICAL THEORIES & ENCYCLOPEDIC RESOURCES

Wikipedia. (n.d.). Newton polynomial. Retrieved February 2026, from https://en.wikipedia.org/wiki/Newton_polynomial

Wikipedia. (n.d.). Runge's phenomenon. Retrieved February 2026, from https://en.wikipedia.org/wiki/Runge%27s_phenomenon

SOFTWARE & WOLFRAM LANGUAGE DOCUMENTATION

Wolfram Research. (n.d.). All (Symbol). Wolfram Language & System Documentation Center. https://reference.wolfram.com/language/ref/All.html

Wolfram Research. (n.d.). BSplineBasis (Symbol). Wolfram Language & System Documentation Center. https://reference.wolfram.com/language/ref/BSplineBasis.html

Wolfram Research. (n.d.). Do (Symbol). Wolfram Language & System Documentation Center. https://reference.wolfram.com/language/ref/Do.html

Wolfram Research. (n.d.). If (Symbol). Wolfram Language & System Documentation Center. https://reference.wolfram.com/language/ref/If.html

Wolfram Research. (n.d.). Mathematical Functions (Guide). Wolfram Language & System Documentation Center. https://reference.wolfram.com/language/guide/MathematicalFunctions.html

Wolfram Research. (n.d.). Real Polynomial Systems (Tutorial). Wolfram Language & System Documentation Center. https://reference.wolfram.com/language/tutorial/RealPolynomialSystems.html

Wolfram Research. (n.d.). Sum (Symbol). Wolfram Language & System Documentation Center. https://reference.wolfram.com/language/ref/Sum.html

Wolfram Research. (n.d.). Table (Symbol). Wolfram Language & System Documentation Center. https://reference.wolfram.com/language/ref/Table.html

TECHNICAL COMMUNITY & SUPPORT RESOURCES

Mathematica Stack Exchange. (2020). Plotting basis order function. [Technical discussion on implementing spline basis functions]. Retrieved from https://mathematica.stackexchange.com/questions/219130/plotting-basis-order-function